

Behavioral Specification Scaffolding for Black-Box Program Reconstruction Agents

ProgramBench proposal: agent-interface design, uncertainty-guided probing, and oracle-grounded repair

Prepared: June 20, 2026 | Current scope: agent scaffolding, not model training

Executive Summary

ProgramBench asks agents to rebuild a program from only a compiled executable and documentation. Our proposal is not to train a new model, but to design a better agent-side interface for this setting: a behavioral specification scaffold that turns black-box executable observations into structured, implementation-ready evidence for a coding agent.

Item	Summary
Core claim	ProgramBench failures are partly specification-recovery failures, not only code-generation failures.
Proposed contribution	A behavioral specification scaffold that builds a behavior map, selects informative probes, and returns oracle-grounded counterexamples.
MVP	Compare the same baseline agent with and without the scaffold on 5-10 easy ProgramBench CLI tasks.
Not current scope	SFT, RL, reward-model training, or full-benchmark scaling. These are future work.

Problem Setting

In ProgramBench, the agent sees a reference executable and task documentation, but not the original source code. It must produce a fresh source codebase whose built executable matches the reference behavior under hidden tests.

- Inputs available to the agent: documentation, reference executable, and local command execution.
- Inputs not available: original source code, hidden tests, internet access during inference.
- Observable behavior: stdout, stderr, exit code, file effects, crashes, and timeouts.
- Main failure mode: agents often start coding before they have enough behavioral evidence.

- Research opportunity: design the agent-computer interface for behavioral reverse engineering, not just generic coding.

Why This Is More Than Probe Generation

A simple pipeline that generates many tests and dumps logs to the model is unlikely to be a strong research contribution. The stronger framing is to study how a scaffold should convert black-box observations into a compact behavioral specification that helps the agent make implementation decisions under limited query, token, and repair budgets.

Basic version	Research-grade version
Run many probes.	Select probes based on uncertainty and information gain: what behavior is still ambiguous?
Dump stdout/stderr logs.	Infer a behavior map: modes, input classes, output templates, error conventions, and file effects.
Report mismatches.	Return oracle-grounded counterexamples with inferred rules and likely implementation implications.
Ask the agent to fix everything.	Prioritize repair targets and break repeated failure loops with new targeted probes.

Proposed Scaffold

The scaffold is a small agent-side component. It does not replace the model. It acts as a ProgramBench-specific agent-computer interface: it decides what to observe, how to summarize observations, and how to feed failures back to the coding agent.

Component	Role
Behavior map	Organize observed behavior by CLI mode, input class, output template, error mode, and file effect.
Uncertainty-guided prober	Generate the next probes to disambiguate behavior that is still unclear or under-covered.

Spec synthesizer	Compress observations into an implementation-oriented partial specification for the agent.
Counterexample miner	Compare reference and candidate, then attach inferred rules to representative failures.
Deadlock breaker	Detect repeated repair loops and request new targeted observations instead of more blind edits.

Agent Loop

1. Initialize a behavior map from docs, help text, and basic executable probes.
2. Identify uncertain behavior regions: undocumented flags, input formats, error handling, file effects, and output templates.
3. Generate high-information probes that distinguish competing behavior hypotheses.
4. Run the reference executable and update the behavior map with oracle-grounded observations.
5. Synthesize a compact behavioral specification and implementation plan for the coding agent.
6. Build and run the candidate program on the same public probes.
7. Mine counterexamples, cluster mismatches, and guide targeted repairs until the fixed budget is exhausted.

MVP Experiment Design

Axis	Plan
Task set	5-10 easy ProgramBench CLI tasks; avoid synthetic sample tasks.
Primary comparison	Baseline agent vs. the same agent plus behavioral specification scaffold.
Control	Same model, task budget, inference setting, and no-internet constraint.
Primary metrics	Hidden-test pass rate, full solve rate, and 95% tests-passed rate.
Secondary metrics	Compile success, public probe match rate, regressions, reference queries, token cost, and wall-clock time.
Diagnostic metrics	Behavior-map coverage, uncertainty reduction, counterexample reuse, and mismatch-cluster reduction.
Success signal	Scaffold improves hidden or partial behavior metrics without simply spending much more budget.

Ablation Plan

- A: baseline mini-swe-agent.
- B: baseline plus static documentation/help summary.
- C: baseline plus behavior map from fixed probes.
- D: baseline plus uncertainty-guided probing.
- E: baseline plus oracle-grounded counterexample repair.
- F: full scaffold: behavior map + uncertainty-guided probing + counterexample repair + deadlock breaker.

The first serious comparison should be A vs. F. Component ablations should follow only if the full scaffold shows a signal.

Positioning Against Recent Agent Work

Prior direction	Takeaway for this project
SWE-agent / agent-computer interfaces	Agent performance depends strongly on the interface and workflow, not only the base model.
Agentless-style controlled pipelines	A simple, inspectable scaffold can be a serious baseline if it isolates the right bottleneck.
Cost-aware SWE agents	Exploration must be budget-aware; more probes are not automatically better.
Agent-generated testing studies	Tests generated by agents can add cost without improving success; our probes are anchored by a real reference executable.
Failure-analysis work	Agents often loop on the same wrong hypothesis; targeted counterexamples can break repair deadlocks.

Baseline Agents and Models

Baseline	Why use it	Role
mini-swe-agent	Official ProgramBench baseline interface; outputs are compatible with programbench eval.	Primary harness baseline.
GPT-5.5	OpenAI's current frontier model for complex reasoning and coding.	Strong closed-model track.
Claude Opus 4.8 / Sonnet 4.6	Current Anthropic models positioned for complex reasoning and agentic coding.	Strong and balanced closed-model tracks.
Gemini 3.1 Pro	Google DeepMind positions it as capable in agentic coding and tool use.	Alternative frontier baseline.
GPT-5.4 mini / Claude Haiku 4.5 / Gemini Flash	Cheaper or weaker models test whether scaffolding helps less capable agents more.	Cost-sensitive track.

Current Local Evidence

A local toy scaffold was migrated into this workspace and validated on `/usr/bin/wc`. This is not a ProgramBench score; it only confirms that the behavior-recording and mismatch-summary machinery works locally.

Item	Summary
Probe cases	21 generated black-box CLI cases.
Candidate	A partial Python reimplementaion of <code>wc</code> .
Result	17/21 exact matches; mean partial behavior score = 0.9524.
Failure cluster	<code>stderr</code> formatting for invalid/help flags.

Infrastructure Constraint

The current machine is a MacBook. Official ProgramBench containers are built for linux/amd64, so full baseline reproduction and official evaluation should run on a Linux x86-64 host. The MacBook is suitable for scaffold development, toy validation, task metadata inspection, and document/prompt design.

Next Steps

- Set up a Linux x86-64 environment with ProgramBench and mini-swe-agent.
- Select 5-10 easy CLI tasks and record a clean baseline run.
- Build the task-level behavior-map and uncertainty-guided probing runner.
- Run scaffold-assisted agents under the same budget as the baseline.
- Compare hidden scores, partial behavior scores, behavior-map coverage, cost, and qualitative trajectories.

Sources

- ProgramBench GitHub: <https://github.com/facebookresearch/programbench>
- ProgramBench paper: <https://arxiv.org/abs/2605.03546>
- SWE-agent paper: <https://arxiv.org/abs/2405.15793>
- Agentless paper: <https://arxiv.org/abs/2407.01489>
- mini-swe-agent ProgramBench docs: <https://mini-swe-agent.com/latest/usage/programbench/>
- OpenAI model docs: <https://developers.openai.com/api/docs/models>
- Anthropic Claude model docs: <https://docs.anthropic.com/en/docs/about-claude/models/overview>
- Google DeepMind Gemini Pro docs: <https://deepmind.google/models/gemini/pro/>